CyberGuard Hub

An Interactive Cybersecurity Awareness Platform

Project Domain	Cybersecurity Awareness
Tech Stack	React (Vite) · Redux Toolkit · React Router · Axios
Styling	Tailwind CSS
Deployment	Vercel
Semester	First Semester — Web Development
Date	May 2026

1. Problem Definition

Cybersecurity threats are growing at an alarming rate, yet most people — especially students and young professionals — remain unaware of how to protect themselves online. While corporate security teams have dedicated tools, the average internet user lacks access to simple, engaging, and up-to-date resources that explain threats in plain language.

There is no single platform that combines real-time threat news, interactive quizzes, a password strength checker, phishing simulation awareness, and a personal threat-tracking dashboard — all in one modern web interface.

Unique Project Statement

"CyberGuard Hub" is a full-stack React web application that empowers everyday users to understand, track, and defend against cybersecurity threats. The platform aggregates live cybersecurity news via external APIs, offers role-based dashboards (Admin / Regular User), interactive quizzes with score tracking, a real-time password strength analyser, and a debounced threat-search — all wrapped in a sleek dark-mode interface with chart-based analytics.

Core Problem Areas Addressed

- Lack of awareness: Most users cannot identify phishing emails or weak passwords.
- Information overload: Cybersecurity news is scattered across dozens of sources.
- No engagement layer: Existing resources are static and non-interactive.
- Zero personalisation: Users get no insight into their own security posture.

2. User Interface (React-Based)

Design Language

- Dark theme by default with a one-click light/dark mode toggle.
- Colour palette inspired by terminal UIs — teal accents on a near-black canvas.
- Responsive layout using Tailwind CSS utility classes — mobile-first design.
- Micro-animations on card hover, quiz transitions, and loading skeletons.

Key Pages / Routes

Route	Purpose
`/`	Landing page with animated hero, threat stats counter, and CTA
`/dashboard`	User dashboard — threat feed, quiz progress, security score chart
`/news`	Paginated cybersecurity news feed with search + filter + sort
`/quiz`	Multi-step cybersecurity quiz with validation and score history
`/tools`	Password checker, phishing URL scanner (simulated)

`/admin`	Admin-only: manage quizzes, view all user scores, role control
`/login`	Auth page — third-party login (Google / GitHub via Auth0)
`/profile`	User profile with edit form and badge showcase

3. State Management

Redux Toolkit — Global Slices

- **authSlice** — Stores user identity, role (admin/user), and JWT token.
- **newsSlice** — Holds fetched articles, pagination state, filters, and sort order.
- **quizSlice** — Tracks current question index, answers, timer, and final score.
- **uiSlice** — Controls dark/light mode, sidebar open state, toast notifications.
- **dashboardSlice** — Aggregates security score, quiz history, and badge data.

Context API — Supplementary Use

- ThemeContext: Propagates colour theme without prop drilling.
- UserPreferenceContext: Stores layout density and notification settings.

4. API / Data Integration

External APIs

API	Usage	Method
NewsData.io / NewsAPI	Real-time cybersecurity news articles	`Axios + async/await`
HaveIBeenPwned API	Check if an email has been in a data breach	`Fetch API`
Auth0	Third-party OAuth login (Google, GitHub)	`Auth0 React SDK`
Random User API	Mock user generation for demo mode	`Axios`
IPInfo API	Geo-locate simulated threat origins	`Fetch API`

Debounced Search Implementation

The news search bar uses a 400 ms debounce via a custom `useDebounce` hook, preventing unnecessary API calls on every keystroke and reducing server load significantly.

5. CRUD Operations

Since the project uses a public REST API (NewsData.io) rather than a custom backend, full CRUD is implemented through a combination of API calls and local Redux state persistence (localStorage-backed).

Operation	Implementation
Create	User can save articles to a personal 'Bookmarks' list; Admin can add new quiz questions.
Read	News feed fetches paginated articles; Dashboard reads quiz scores and saved items.
Update	User can edit their profile info (name, bio, avatar URL) via multi-step form.
Delete	User can remove bookmarked articles; Admin can delete quiz questions from the panel.

6. Performance Optimization

- **React.lazy + Suspense:** Dashboard, Admin Panel, and Quiz pages are code-split and lazy-loaded.
- **useMemo / useCallback:** Expensive filtering and sorting computations are memoised in the news feed.
- **Infinite Scroll:** The news page uses IntersectionObserver to load the next page as the user scrolls.
- **Image lazy loading:** Article thumbnail images use the native `loading='lazy'` attribute.
- **Debounced API calls:** Custom hook prevents redundant network requests from the search bar.
- **Redux selector memoisation:** `reselect` library used for derived data selectors on the dashboard.

7. Error Handling

Error Boundary (Class Component)

A top-level `ErrorBoundary` component wraps the entire application. A secondary boundary wraps the Dashboard widget grid — ensuring a broken chart does not crash the whole page. Each boundary renders a styled fallback UI with a retry button.

API Error Strategy

- All Axios calls are wrapped in try/catch with specific HTTP status handling.
- 401 responses trigger automatic logout and redirect to /login.
- Network failures show a toast notification with a 'Retry' action.
- Empty API responses render a dedicated empty-state illustration.

8. Advanced Feature Set (5 Implemented)

Feature	Description	Priority

Auth + RBAC	Auth0 OAuth login; Admin role sees extra routes and controls	**Core**
Pagination + Inf. Scroll	News page uses infinite scroll; Quiz history uses numbered pages	**Core**
Search + Filter + Sort	Debounced search, category filter, and date/relevance sort on news	**Core**
Dark Mode Toggle	System-preference detection on load; persisted in localStorage	**Core**
Dashboard + Charts	Recharts bar/line charts for quiz scores, security grade over time	**Advanced**
Debounced API Calls	Custom useDebounce hook — 400 ms delay on search input	**Advanced**
Error Boundary	Two-level boundary strategy with graceful fallback UI	**Advanced**
Memoisation	useMemo, useCallback, and reselect across perf-critical paths	**Advanced**
Multi-step Form	Profile edit and quiz setup use multi-step form with Zod validation	**Advanced**

9. Project Folder Structure

```
src/
├── api/
│   ├── newsApi.js          # Axios instance + news endpoints
│   ├── authApi.js          # Auth0 config and helpers
│   └── breachApi.js        # HaveIBeenPwned integration
├── app/
│   └── store.js            # Redux store setup
├── components/
│   ├── common/             # Button, Modal, Toast, Loader
│   ├── layout/             # Navbar, Sidebar, Footer
│   └── charts/             # Recharts wrappers
├── features/
│   ├── auth/               # authSlice + Auth0 hooks
│   ├── news/               # newsSlice + NewsCard, NewsFilter
│   ├── quiz/               # quizSlice + QuizEngine
│   └── dashboard/          # dashboardSlice + widgets
├── hooks/
│   ├── useDebounce.js
│   ├── useInfiniteScroll.js
│   └── useLocalStorage.js
├── pages/
│   ├── Home.jsx
│   ├── Dashboard.jsx (lazy)
│   ├── News.jsx (lazy)
│   ├── Quiz.jsx (lazy)
│   ├── Tools.jsx
│   ├── Admin.jsx (lazy, protected)
│   └── Profile.jsx
├── router/
│   ├── AppRouter.jsx       # React Router v6 setup
│   └── ProtectedRoute.jsx  # Role-based guard
├── utils/
│   ├── passwordStrength.js
│   └── formatDate.js
└── main.jsx
```

10. Deployment & Documentation

Vercel Deployment

- Repository pushed to GitHub; Vercel connected for automatic CI/CD on every push to main.
- Environment variables (API keys, Auth0 credentials) stored in Vercel project settings.
- `vite.config.js` configured for production build with code-splitting.

Documentation

README.md	Setup instructions, env variable guide, local dev commands, live demo link
JSDoc Comments	All utility functions and custom hooks documented with parameter descriptions
Inline Comments	Complex Redux selectors and async thunks annotated inline
Component Props	PropTypes defined for all shared components

11. Tech Stack Summary

React (Vite)	**Redux Toolkit**	**React Router v6**	**Axios**
Tailwind CSS	**Auth0**	**Recharts**	**Vercel**

12. Development Timeline

Week 1–2	Project setup, routing skeleton, Auth0 integration, Redux store
Week 3–4	News API integration, infinite scroll, search/filter/sort
Week 5–6	Quiz engine, multi-step form, score tracking, localStorage
Week 7–8	Dashboard charts, password tool, error boundaries, dark mode
Week 9	Performance audit — memoisation, lazy loading, bundle analysis
Week 10	Deployment, documentation, final testing and README polish